ENGENHARIA DE SOFTWARE • ATIVIDADE AVALIATIVA A2

Sistema de Agendamento de Monitorias

Dupla: Vinicius Gonçalves de Andrade e Lucas Gabriel Rodrigues de Moraes

Entrega: Parte 2 (Projeto, Relatório e Documentação)

Seção 1: Projeto Inicial — História 1: Marcar monitoria

História escolhida (Prioridade Alta)

Como aluno, **eu quero** escolher a disciplina, ver os horários disponíveis e marcar o atendimento informando a minha dúvida, **para** marcar a monitoria de forma fácil e rápida.

Justificativa da escolha: Escolhida por ter prioridade Alta e porque o restante do sistema depende dela.

Estrutura de classes e funções

O sistema usa uma classe base e uma classe por tipo de usuário, com herança, além de uma classe pequena para o horário. Nada depende de dados digitados durante a execução: os métodos recebem e devolvem valores, para que os testes os chamem diretamente. O código fica na pasta `codigo/`, com um arquivo para as classes e outro para os testes com `unittest`.

Classe Usuario (base): Guarda o nome. É a classe da qual os tipos de usuário herdam.

Classe Aluno (herda de Usuario): Guarda se o aluno está bloqueado. É quem executa a marcação.

- `marcar_monitoria(horario, duvida)`: verifica se o aluno está bloqueado e se o horário está disponível. Se estiver tudo certo, registra a marcação salvando a dúvida. Caso contrário, lança uma exceção com mensagem clara.

Classe Monitor (herda de Usuario): Guarda a disciplina que atende e a sua lista de disponibilidades.

- `adicionar_disponibilidade(data_hora)`: cria um horário de 30 minutos para o monitor e o guarda na sua lista.
- `horarios_disponiveis()`: devolve somente os horários livres do monitor.

Classe Horario: Representa um horário de 30 minutos de um monitor. Guarda a disciplina, o monitor, a data e hora, o aluno que marcou (se houver) e a dúvida informada.

- `esta_disponivel()`: diz se o horário ainda está livre.

Função listar_horarios_disponiveis(monitores, disciplina): Percorre os monitores da disciplina escolhida e junta os horários livres de todos. É o que o aluno usa para "ver os horários da disciplina".

Exceções: Uma para horário já ocupado (`HorarioOcupadoError`) e outra para aluno bloqueado (`AlunoBloqueadoError`).

Critérios de aceitação e testes previstos no Projeto Inicial

Critério de Aceitação	Teste (`unittest`)	Cenário
O aluno escolhe a disciplina e vê apenas os horários disponíveis dela	`test_lista_somente_horarios_livres_da_disciplina`	Sucesso
O aluno informa a dúvida ao marcar, e ela fica salva	`test_marcacao_salva_a_duvida`	Sucesso
Depois de marcado, o horário deixa de aparecer como disponível	`test_horario_marcado_nao_aparece_na_lista`	Sucesso
O sistema não permite marcar um horário já ocupado	`test_marcar_horario_ocupado_lanca_excecao`	Falha

Seção 2: Relatório de Execução (README)

Instruções para executar o sistema e os testes

- **Versão do Python:** Python 3.10 ou superior (testado em Python 3.14).
- **Bibliotecas necessárias:** Nenhuma biblioteca externa é necessária. O protótipo utiliza apenas módulos da biblioteca padrão do Python (`datetime` e `unittest`).
- **Arquivo principal do sistema:** `codigo/monitoria.py`

Comando para executar o sistema (a partir da pasta raiz do projeto):

```
python codigo/monitoria.py
```

Comando para executar os testes automatizados:

```
python -m unittest discover -s codigo
```

Comparação entre o Plano de Execução e o Realizado

Data	Responsável	O que foi planejado em aula	O que foi realizado na execução
17/10 (Sábado)	Vinicius	Implementação das classes `Coordenacao` , `Professor` , `Monitor` e `Aluno` .	Concluído na data prevista. Classes implementadas com herança a partir da classe base `Usuario` em `codigo/monitoria.py` .
17/10 (Sábado)	Lucas	Sistema de marcação de monitorias com as classes de monitor/aluno, salvando dúvida e horário.	Concluído na data prevista. Marcação de monitoria, listagem por disciplina e os 5 testes de aceitação da História 1 implementados.
20/10 (Terça)	Vinicius	Acesso exclusivo da coordenação p/ relatório de horas e verificação de dúvidas para professores e monitores.	Concluído na data prevista. Criados os arquivos `acesso.py` , `relatorios.py` e `duvidas.py` com controle de acesso e consultas.
20/10 (Terça)	Lucas	Sistema de cancelamento de monitoria para alunos e bloqueio por faltas.	Concluído na data prevista. Implementados `cancelar_monitoria` (regra de 2h antes), `registrar_falta` (bloqueio com 2 faltas) e testes.

Atrasos e mudanças em relação ao Projeto Inicial

Não houve atrasos em relação ao cronograma planejado pela dupla em aula: cada etapa foi entregue no dia previsto. Em relação ao escopo descrito no Projeto Inicial (que detalhava apenas a estrutura mínima para atender à História 1), foram feitas as seguintes evoluções justificadas durante o desenvolvimento:

- **Adição do cancelamento e controle de faltas em `monitoria.py` :** Para cumprir a etapa planejada para o Lucas em 20/10 (História 2), adicionamos a exceção `CancelamentoForaDoPrazoError` , o atributo `faltas` em `Aluno` e o método `cancelar_monitoria(horario, hora_atual)` usando `datetime` para validar o limite mínimo de 2 horas de antecedência, além de `cancelar_horario` na classe `Monitor` .
- **Criação das classes `Professor` e `Coordenacao` e separação em módulos (`acesso.py` , `duvidas.py` e `relatorios.py`):** Para cumprir as entregas do Vinicius planejadas para 17/10 e 20/10 sem sobrecarregar o arquivo principal da História 1, organizamos o controle de permissão por tipo de usuário, o agrupamento/ranking de dúvidas e o cálculo mensal de horas da bolsa em módulos separados dentro da pasta `codigo/` .

Seção 3: Documentação Técnica

1. Histórias de usuário

História 1: Marcar monitoria (História Principal Implementada)

Prioridade: Alta

Como aluno, **eu quero** escolher a disciplina, ver os horários disponíveis e marcar o atendimento informando a minha dúvida **para** marcar a monitoria de forma fácil e rápida, sem combinar por mensagem com o monitor.

Critérios de aceitação:

- O aluno escolhe a disciplina e vê apenas os horários disponíveis dela.
- O aluno informa a dúvida ao marcar, e ela fica salva junto com a marcação.
- Depois de marcado, o horário deixa de aparecer como disponível.
- O sistema não permite marcar um horário que já está ocupado.
- O aluno bloqueado não consegue marcar monitoria.

Justificativa: Sem a marcação funcionando o sistema não tem utilidade, e o cliente afirma que, se a reserva de horários funcionar, já ajuda muito.
Trecho do e-mail: *"A ideia é que o aluno escolha a disciplina, veja os horários livres e reserve um atendimento."*

História 2: Cancelar monitoria e controlar faltas

Prioridade: Média

Como aluno, **eu quero** cancelar a minha monitoria até 2 horas antes do atendimento **para** não perder o horário e não ser bloqueado por faltar.

Critérios de aceitação:

- O aluno cancela a própria monitoria até 2 horas antes do atendimento.
- Depois do cancelamento, o horário volta a aparecer como disponível.
- O sistema não permite cancelar com menos de 2 horas de antecedência.
- O aluno que não cancela e não comparece recebe uma advertência (falta).
- O aluno com 2 advertências é bloqueado e não consegue marcar novas monitorias.

Justificativa: Reduz os horários perdidos por falta, mas depende de a marcação (história 1) já funcionar.
Trecho do e-mail: *"Quando o aluno reserva e não aparece, o horário fica perdido. Alguns monitores sugeriram bloquear quem falta muito, mas outros acham isso exagerado."*

História 3: Gerenciar horários e atendimentos em grupo

Prioridade: Alta

Como monitor, **eu quero** informar os horários em que posso atender, cancelar um horário quando precisar (por exemplo, por causa de prova) e marcar atendimento em grupo quando alunos tiverem a mesma dúvida **para** que os alunos saibam quando há atendimento, ninguém chegue à sala à toa e eu resolva a dúvida de vários alunos de uma vez.

Critérios de aceitação:

- O monitor cadastra horários de atendimento de 30 minutos na sua disciplina, que aparecem como disponíveis para os alunos.
- O monitor cancela um horário seu, que deixa de aparecer como disponível.
- Os alunos que tinham marcado o horário cancelado são avisados do cancelamento.
- O monitor vê as dúvidas dos alunos que marcaram atendimento com ele e o sistema mostra quando mais de um aluno tem a mesma dúvida.
- Somente o monitor decide marcar o atendimento como em grupo.

Justificativa: A marcação (história 1) depende de os monitores terem informado seus horários, e o cancelamento e o atendimento em grupo resolvem problemas relatados pelo cliente.
Trecho do e-mail: *"cada monitor informa os horários em que pode atender, mas às vezes precisa cancelar por causa de prova. Os alunos reclamam que só descobrem o cancelamento quando chegam na sala."* e *"alguns monitores preferem atender em grupo, quando várias pessoas têm a mesma dúvida"*.

1. Histórias de usuário (continuação)

História 4: Ver as maiores dúvidas dos alunos

Prioridade: Baixa

Como professor, **eu quero** ver quais são as maiores dúvidas dos alunos que vão às monitorias da minha disciplina **para** melhorar a minha aula e reforçar os conteúdos mais difíceis.

Critérios de aceitação:

- O professor vê as dúvidas dos alunos que marcaram monitoria na sua disciplina.
- A lista mostra primeiro as dúvidas que aparecem mais vezes.
- O professor só vê as dúvidas das suas próprias disciplinas.
- Se não houve monitorias, a lista aparece vazia, sem erro.

Justificativa: Ajuda o ensino em sala, mas o atendimento funciona sem isso.
Trecho do e-mail: *"Os professores das disciplinas gostariam de saber quais conteúdos geram mais dúvidas, para reforçar em aula."*

História 5: Relatório mensal de horas

Prioridade: Média

Como coordenação, **eu quero** um relatório mensal com as horas atendidas por cada monitor **para** calcular o pagamento da bolsa.

Critérios de aceitação:

- O relatório mostra, por monitor, o total de horas atendidas no mês.
- Somente a coordenação consegue gerar o relatório.
- Alunos, monitores e professores não têm acesso ao relatório.
- Um mês sem atendimentos gera relatório com zero horas.

Justificativa: É necessário para o pagamento da bolsa, mas só faz sentido depois que os atendimentos já existirem.
Trecho do e-mail: *"A coordenação precisa de um relatório mensal com as horas atendidas por cada monitor, porque isso conta para o pagamento da bolsa. Essas informações de bolsa não podem ficar visíveis para todo mundo."*

Síntese das Histórias de Usuário por Stakeholder e Módulo

História	Tipo de Usuário	Prioridade	Arquivo no Código	Status no Protótipo
História 1	Aluno	Alta	`codigo/monitoria.py`	Implementada e testada
História 2	Aluno	Média	`codigo/monitoria.py`	Implementada e testada
História 3	Monitor	Alta	`monitoria.py` / `duvidas.py`	Lógica implementada
História 4	Professor	Baixa	`codigo/duvidas.py`	Lógica implementada
História 5	Coordenação	Média	`codigo/relatorios.py`	Lógica implementada

2. Testes de aceitação

Todos os testes de aceitação foram implementados no arquivo `codigo/test_monitoria.py` utilizando o módulo `unittest`, cobrindo integralmente a História 1 (história principal escolhida) e os casos de cancelamento e bloqueio por faltas da História 2.

História	Critério de Aceitação Verificado	Nome do Caso de Teste (`test_monitoria.py`)	Cenário
História 1 (Principal)	O aluno escolhe a disciplina e vê apenas os horários disponíveis dela.	`test_lista_somente_horarios_livres_da_disciplina`	Sucesso
História 1 (Principal)	O aluno informa a dúvida ao marcar, e ela fica salva junto com a marcação.	`test_marcacao_salva_a_duvida`	Sucesso
História 1 (Principal)	Depois de marcado, o horário deixa de aparecer como disponível.	`test_horario_marcado_nao_aparece_na_lista`	Sucesso
História 1 (Principal)	O sistema não permite marcar um horário que já está ocupado.	`test_marcar_horario_ocupado_lanca_excecao`	Falha / Exceção (`HorarioOcupadoE`
História 1 (Principal)	O aluno bloqueado não consegue marcar monitoria.	`test_aluno_bloqueado_nao_marca`	Falha / Exceção (`AlunoBloqueadoE`
História 2 (Adicional)	O aluno cancela a monitoria até 2h antes e o horário volta a aparecer como disponível.	`test_cancelar_monitoria_dentro_do_prazo`	Sucesso
História 2 (Adicional)	O sistema não permite cancelar com menos de 2 horas de antecedência.	`test_cancelar_monitoria_fora_do_prazo_lanca_excecao`	Falha / Exceção (`CancelamentoFora`